

What we store, and how it gets there

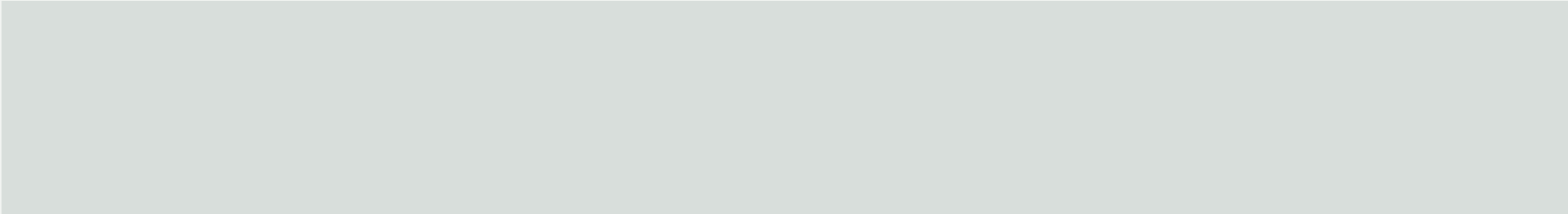
Every clinic gets its own Supabase project running one identical schema. Thirty-one tables hold three different kinds of thing: the clinic's *rulebook*, set once at onboarding; the *live record* of leads, messages and appointments, written continuously by twenty-three n8n workflows; and the *weekly read*, written once a week by the analysis pipeline. This is the full list, what each one is for, and which workflow puts data in it.

31 TABLES + 2 VIEWS	23 WORKFLOWS WIRED	7 WAYS DATA ENTERS	47 EVENT TYPES LOGGED	0 RAW PHONE NUMBERS
---	--	--	---	---

01 — INTAKE

The seven ways data gets in

Nothing is typed into the database by hand after onboarding. Everything below is either a live event pushed to us, a clock firing on a schedule, or a batch we import deliberately.



PUSH · LIVE A patient messages on WhatsApp Meta posts every inbound message to a webhook. This is the single busiest door — it starts conversations, detects replies, catches opt-outs, and confirms deliveries.	PUSH · LIVE A booking is made or changed The clinic's booking system posts the appointment to a webhook, which deduplicates it and files it against the lead.	PUSH · LIVE Someone moves a lead in the sheet When a rep drags a lead into "Proposal Sent", a Google Sheets script relays that edit to us and the follow-up sequence starts.
PUSH · LIVE The voice agent takes a call The AI receptionist calls out to us mid-conversation to look a caller up, create them, or hand the call to a human.	BATCH · MANUAL A CSV of dormant leads is uploaded Old enquiries from the clinic's CRM are imported in bulk, cleaned, checked against the do-not-contact list, and enrolled.	CLOCK · EVERY 3-15 MIN Scheduled jobs wake up Ten separate crons: send reminders, fire the next follow-up touch, detect no-shows, watch response times, retry failures, flush messages queued overnight.
CLOCK · WEEKLY The analysis pipeline runs Once a week a separate Python pipeline reads everything above, scores it, and writes the diagnosis back into its own set of tables.		

The register

Grouped by what the data *is*, not by which service uses it — most tables are shared across services. Each entry shows what sits in it, how it gets there, and the question it exists to answer.

A · The clinic's rulebook

Filled in once at onboarding. Read constantly, written almost never.

Clinic profile	STORED
	Clinic name, their Google review link, and how many hours after a treatment we should ask for a review.
	HOW IT LANDS
client_config SET ONCE	Entered at onboarding. Read by the appointment, reminder and review workflows on every run.
	WHY WE NEED IT
	Keeps anything clinic-specific out of the workflow itself, so a second clinic can never be sent the first clinic's review link.

The stages a lead moves through	STORED
	Ten named stages in order — New Inquiry, Owner Notified, First Contact Made, In Conversation, Appointment Scheduled, Show, No Show, Enrolled, Not Interested, Sequence Exhausted — and which of them are dead ends.
	HOW IT LANDS
pipeline_stages SET ONCE	Seeded once with the schema. Every lead points at exactly one of these names.
	WHY WE NEED IT
	This is what makes "where are leads dying?" a question with an answer. Without a fixed, ordered list, funnel drop-off can't be measured.

Staff

STORED

Name, WhatsApp number, whether they're currently taking leads, how many they're holding right now, their daily cap, and whether they're a rep or a manager.

HOW IT LANDS

Added at onboarding, then updated automatically each time a lead is assigned — last assigned time, queue count up by one.

reps SET ONCE

WHY WE NEED IT

Three jobs: fair round-robin distribution, knowing who to escalate to, and per-person accountability in the weekly report. The manager flag is who gets told about bad feedback.

Response-time rules

STORED

Per stage: how many minutes before a lead counts as neglected, how many before it escalates, and the clinic's real operating hours and working days.

HOW IT LANDS

Agreed with the clinic at onboarding. Read by every workflow before it sends anything.

sla_config SET ONCE

WHY WE NEED IT

"Responded in five minutes" is meaningless without a target. The operating hours matter just as much — they stop us counting 9pm silence as a failure, or messaging a patient at midnight.

Legacy response rules

STORED

An older, coarser version of the same thing — thresholds in hours, with no operating-hours window and no per-clinic split.

HOW IT LANDS

Seeded historically. Nothing currently writes to it.

sla_rules SET ONCE

WHY WE NEED IT

Kept for backward compatibility only. New work should use the config table above; this one is a candidate to retire.

Follow-up playbooks

STORED

A named sequence per service — reactivation, quote follow-up, no-show recovery — with how many touches it has, how far apart they land, and the actual message templates for each touch.

HOW IT LANDS

Seeded per service before any workflow runs. The sender reads it fresh on every touch.

sequences

SET ONCE

WHY WE NEED IT

Changing the cadence or rewriting a message is a data edit, not a workflow rebuild. It also means each touch can carry a genuinely different message instead of the same nudge five times.

Stop words

STORED

Phrases that mean "stop messaging me" — stop, unsubscribe, not interested, remove me — each tagged with a language and how strictly to match it (exact, contains, starts with).

HOW IT LANDS

Seeded, then extended over time. Checked against every single inbound message before anything else happens.

opt_out_phrases

SET ONCE

WHY WE NEED IT

Consent compliance, and it has to work in the languages patients actually reply in — the list carries Hindi, Telugu, Tamil, Marathi and Kannada as first-class options, not just English.

Treatment menu

STORED

The list of treatments the WhatsApp bot offers: display name, order in the menu, price, and how many intent points picking it is worth. Can nest two levels deep for clinics with long menus.

HOW IT LANDS

Seeded per clinic from their actual service list.

intake_services

SET ONCE

WHY WE NEED IT

The bot's menu is data, so every clinic gets their own without anyone touching the workflow. The price and points are what let an enquiry be valued and ranked at the moment it arrives.

Qualifying questions

STORED

Per treatment, the questions the bot asks, the multiple-choice answers offered, the points each answer carries, and how the option should be presented.

HOW IT LANDS

Seeded per clinic per treatment.

intake_questions

SET ONCE

WHY WE NEED IT

Turns a vague enquiry into structured answers — timeline, budget, urgency — so the lead arrives at a human already qualified, and so scoring is comparable between leads.

Hot / warm / cold thresholds

STORED

Two numbers per clinic: the score at which a lead counts as hot, and the score at which it counts as warm.

HOW IT LANDS

Seeded per clinic, tuned once real volume exists.

SET ONCE

WHY WE NEED IT

One clinic's "hot" is another's "maybe". Keeping the cutoff in data means we can retune it from evidence instead of guessing at build time.

Canned answers

STORED

Topic, question, and the approved answer, with an on/off switch.

HOW IT LANDS

Written with the clinic during onboarding.

faq_entries

SET ONCE

WHY WE NEED IT

The bot answers the predictable questions — parking, timings, does it hurt, do you do EMI — in the clinic's own words, without escalating to a human.

Clinic knowledge base

STORED

Chunks of the clinic's own written material, each paired with a mathematical fingerprint that lets us find the passage most relevant to a question.

HOW IT LANDS

Loaded at onboarding from clinic brochures, site copy and treatment notes.

clinic_documents

SET ONCE

WHY WE NEED IT

When a patient asks something off-script, the bot answers from the clinic's real material instead of inventing something. This is the difference between a helpful assistant and a liability.

Whose turn it is

STORED

A single number per team: the position in the staff list that the next lead goes to.

HOW IT LANDS

Incremented by the handoff workflow every time a lead is assigned.

assignment_counter

WHY WE NEED IT

LIVE · WORKFLOWS

Fair distribution that survives two leads arriving in the same second. Keeping the pointer in the database rather than in workflow memory is what stops both being handed to the same person.

B · The people

Who enquired, where they came from, and whether we're still allowed to contact them.

Leads

STORED

The master record for a person. Scrambled phone and email (never the real thing), which channel they came from, the value of the treatment they want, how hot they are, the stage they're at, won or lost with a reason, who owns them, when they were last contacted in each direction, whether they're reachable on WhatsApp or email, their consent and marketing-consent state, whether they've opted out, whether they're eligible for reactivation, and their ids in the clinic's CRM. Plus a free-form pocket for anything a particular import brings along.

HOW IT LANDS

Created by whichever door they came through — the intake bot, a CSV import, a booking webhook, or the voice agent — then updated by almost every workflow in the system for the rest of their life.

WHY WE NEED IT

It's the spine; everything else hangs off it. The one field worth calling out is the deal value: because it's recorded per lead rather than averaged, revenue leakage can be calculated from what was actually at stake instead of a blended guess.

Where they came from

STORED

Platform (Instagram, Facebook, WhatsApp, website, direct), the entry point (lead ad, click-to-WhatsApp, organic DM, web form, missed call, walk-in), then the exact ad, ad set and campaign — both id and name — plus form id, placement, click ids, all five UTM tags, the landing page, and the untouched original payload.

HOW IT LANDS

Written the moment someone first makes contact, filed against their scrambled phone number, then linked to their lead record once one exists.

WHY WE NEED IT

To answer "which specific ad produced booked appointments" rather than "which ad produced clicks". Keeping the raw payload means a field we didn't think to capture today can still be recovered later.

Conversations in progress

intake_sessions

LIVE · WORKFLOWS

STORED

The state of a live WhatsApp chat before it has become a lead: which step it's on, which treatment they picked, their answers so far, the running score and hot/warm/cold verdict, how many times they've sent something the bot didn't understand, and the last message ids in each direction.

HOW IT LANDS

Created on the first inbound message, updated on every reply, closed out when it converts into a lead.

WHY WE NEED IT

The bot can pick a conversation back up hours later instead of restarting. It also makes abandonment visible — we can see exactly which question people stop answering at, which is otherwise invisible.

Do-not-contact list

opt_outs

LIVE · WORKFLOWS

STORED

Scrambled phone and email, when they opted out, whether it came from a reply or was set by hand, and which channel it applies to.

HOW IT LANDS

Written the instant a stop phrase is matched on an inbound message. Checked before every outbound send, by every service.

WHY WE NEED IT

One row here silently overrides every sequence in the system. It's deliberately a separate table from the flag on the lead so that an opt-out survives even if the lead record is later merged or replaced.

C · What actually happened

The evidence layer. Every number in every report is derived from these five tables.

The timeline

lead_events

LIVE · WORKFLOWS

STORED

One row for every single thing that happens to a lead, drawn from a fixed vocabulary of 47 event types — created, assigned, notified, first contact made, message sent, reply received, appointment booked, no-show marked, opted out, review requested, and so on. Each row records whether a **human** did it or the system did, who, what state the lead was in at the time, and the supporting detail.

HOW IT LANDS

Every workflow writes here after every meaningful step. It is append-only — rows are never edited or removed.

WHY WE NEED IT

This is the single most important table we have. The human-versus-automated flag is what stops us crediting the clinic for work a bot did, and the fixed vocabulary is what makes the weekly report reproducible rather than a fresh interpretation each week.

Appointments

appointments

LIVE · WORKFLOWS

STORED

When the appointment is, its status (scheduled, showed, no-showed, cancelled, rescheduled, completed), the calendar system's own id, which staff member it belongs to, whether the 24-hour and 1-hour reminders actually went out, the rescheduling link, when a no-show was confirmed, and the whole review cycle — requested, when, submitted, and the rating given.

HOW IT LANDS

Created by the booking webhook, flipped to no-show by a cron that checks past-due appointments after a grace period, and patched by the reminder and review workflows as each step completes.

WHY WE NEED IT

Show rate, no-show rate, how much lift the reminders actually produce, and how many no-shows come back — the entire No-Show Recovery service is computed from this table. The reminder flags are what make "lift" measurable rather than assumed.

Who is in which sequence

sequence_enrollments

LIVE · WORKFLOWS

STORED

The link between a lead and a playbook: status (active, paused, completed, replied, exhausted, terminated), which touch they're on, when the next one is due, a processing lock, when a reply was spotted, and who it was handed to when it was.

HOW IT LANDS

Created when a lead is enrolled, advanced by the ticker every fifteen minutes, and stopped the instant a reply arrives.

WHY WE NEED IT

It's the state machine for every follow-up in the business. The lock in particular earns its place: it's what stops two overlapping runs sending the same person the same message twice.

Every message sent

sequence_touches

LIVE · WORKFLOWS

STORED

One row per message: which touch number it was, the channel and provider, the template name and version, the provider's own message id, and separate timestamps for queued, sent, delivered, read, replied and failed — with the reason on failure.

HOW IT LANDS

Written as "queued" before the send, promoted to "sent" once the provider accepts it, then updated again when delivery and read receipts come back.

WHY WE NEED IT

Proves a message was actually delivered rather than merely attempted — WhatsApp will happily return success for a message it silently drops. It's also what lets a touch that comes due at 11pm be held and fired the next morning instead of lost.

The response clock

sla_tracking

LIVE · WORKFLOWS

STORED

One row per lead: when it was assigned, when the owner was actually *notified* — which is where the clock genuinely starts — the deadline, when they first responded, how many times the deadline was blown, how many real human contact attempts were made versus automated touches, and whether it's been escalated.

HOW IT LANDS

Created when a lead is assigned to a person, then patched by the monitor cron every three minutes and by the reply handler when a response finally lands.

WHY WE NEED IT

This is the Speed-to-Lead service. The split between human attempts and automated touches is the whole point — an automated message going out is not the clinic responding, and counting it as one would make the number a lie.

D · Did the machine actually run *Four tables whose only job is to make silence distinguishable from success.*

Bulk import runs

reactivation_runs

LIVE · WORKFLOWS

STORED

Per CSV upload: the file name and a fingerprint of its contents, rows total, valid, enrolled, skipped and failed — and then *why* each was skipped, broken out: already opted out, already enrolled, no reachable channel, duplicate within the file, invalid data.

HOW IT LANDS

One row per upload, counters incremented as the file is processed.

WHY WE NEED IT

So "we uploaded 800 and enrolled 340" comes with all 460 accounted for. Without the skip breakdown, a bad import looks identical to a clean one with a lot of opt-outs.

Workflow runs

workflow_logs

LIVE · WORKFLOWS

STORED

Which workflow ran, when it started and finished, its run id, what triggered it, the outcome — success, failed, partial, skipped, or ran-but-found-nothing — and how many records it handled.

HOW IT LANDS

Written at the start and end of every scheduled run.

WHY WE NEED IT

Chiefly to separate "nothing happened" from "nothing was supposed to happen". A zero-data run is a healthy outcome; a missing run is not, and they look the same from outside.

Failures

error_logs

LIVE · WORKFLOWS

STORED

Which workflow failed, the type and text of the error, the exact payload that caused it, how many times we've retried, when we'll retry next, and whether it's been resolved.

HOW IT LANDS

Written by any workflow that hits a problem. A maintenance cron then picks these up every five minutes, retries them with backoff, and escalates to a human after three failures.

WHY WE NEED IT

It makes the system partly self-healing, and it gives us a real count of how flaky each integration is — expiring WhatsApp tokens and CRM outages are the two recurring ones, and they're only visible because of this.

Is it still alive

heartbeat

LIVE · WORKFLOWS

STORED

The last time each scheduled workflow checked in, how often it's supposed to, and whether it's now considered stale.

HOW IT LANDS

Each cron pings its own row at the top of every run. A separate watcher checks the pings and raises an alert when one goes quiet.

WHY WE NEED IT

A dead scheduler is completely invisible without this — it produces no errors, no logs, no failed runs. Nothing failing and nothing running look identical until something is expected to arrive and doesn't.

Weekly scorecard

pipeline_runs

WEEKLY · ANALYSIS

STORED

One row per clinic per week: overall health score, response-time compliance, ownership rate, stale rate, lead counts — plus how much of the underlying data was actually computable, whether the run halted and why. Then each service's headline numbers: show and no-show rate, recovery rate, the lift the reminders produced, money at risk, and revenue leakage stored as a low-to-high band rather than a single number.

HOW IT LANDS

One row written per week, replaced if the week is re-run. A service that didn't run leaves its columns empty rather than zero.

WHY WE NEED IT

Week-over-week trend reads straight out of this table instead of being recomputed from scratch. Empty-not-zero matters: it stops a service that didn't run from looking like a service that scored nothing.

The diagnosis

weekly_insights

WEEKLY · ANALYSIS

STORED

Where the pipeline is leaking this week, what pattern it matches, the root cause, the rule we're putting in place to fix it, and the key metrics before and after — with the health-score change calculated automatically so it can't drift.

HOW IT LANDS

Written each week alongside the scorecard. Re-running a week refreshes the numbers without wiping the written analysis.

WHY WE NEED IT

It turns the report from a dashboard into a diagnosis — one named problem with a named fix, and evidence next week of whether the fix worked.

Per-person weekly

owner_performance

WEEKLY · ANALYSIS

STORED

Per staff member per week: leads assigned, how many were contacted inside the response window, how many got zero or barely any effort, how many went stale, and how many needed rescuing.

HOW IT LANDS

Rebuilt from scratch each run — the old rows for that week are cleared and replaced, so re-running can't produce duplicates.

WHY WE NEED IT

Turns a clinic-level number into an accountable one. "Response time is 40%" is an argument; "these three leads sat untouched with this person for six days" is a conversation.

Leads at risk

rescue_flags

WEEKLY · ANALYSIS

STORED

Individual leads flagged for one of three reasons — gone stale, bounced between owners, or moved forward without the prior step ever happening — with their value tier, current owner, attempt count and age.

HOW IT LANDS

Computed weekly, one row per flagged lead per run.

WHY WE NEED IT

It's the actionable half of the report: a specific list of leads that can still be saved this week, rather than a rate that describes leads already lost.

The ten worst-handled leads

worst10_log

WEEKLY · ANALYSIS

STORED

The worst cases of the week with a full post-mortem each: what kind of failure it was, the first point at which it went wrong, a severity score, the deal value and tier, how many times it changed hands, and how confident we are in the diagnosis given the data available.

HOW IT LANDS

Computed weekly from the timeline and the scorecard.

WHY WE NEED IT

Concrete cases are what change behaviour. The confidence field is deliberate — a case built on thin evidence is labelled as such rather than presented with the same certainty as a complete one.

Two rollup views

v_pattern_library · v_client_eng
agement

WEEKLY · ANALYSIS

STORED

Nothing of their own — they're saved questions over the weekly rows. One groups recurring problem patterns and how often each gets resolved; the other tracks how long a clinic has been with us and how their health has moved since the baseline.

HOW IT LANDS

Calculated live whenever they're read.

WHY WE NEED IT

Long-arc questions — "is this the third time we've seen this pattern here?", "has the score actually moved since month one?" — without storing a second copy of anything.

Which workflow touches which data

Twenty-three workflows across six services. Twenty-two of them write; one is read-only. All of them share the same tables. No workflow talks to the database directly — every read and write goes through Supabase's REST layer, which is why the same schema can serve every client without any per-client code.

Lead Qualification 1 workflow • the WhatsApp intake bot

The front door. Greets an inbound message, shows the treatment menu, asks the qualifying questions, scores the answers, creates the lead, and hands it to a person.

Intake Agent

MCI_01

intake_sessions	leads	lead_events	opt_outs	sequence_enrollments	appointments	assignment_counter	intake_services
intake_questions	intake_scoring_rules	opt_out_phrases	reps				

Dead Pipeline Reactivation 4 workflows · dormant leads

Takes a CSV of old enquiries, enrolls them in a five-touch WhatsApp sequence, and hands anyone who replies to a rep in round-robin order.

CSV Ingestion & Enrolment

WF1 · MANUAL

- leads
- sequence_enrollments
- reactivation_runs
- lead_events
- workflow_logs
- sequences
- opt_outs

Sequence Execution

WF2 · EVERY 15 MIN

- sequence_touches
- sequence_enrollments
- leads
- lead_events
- heartbeat
- error_logs
- workflow_logs
- sequences

Reply Detection

WF3 · WEBHOOK

- opt_outs
- leads
- sequence_enrollments
- lead_events
- error_logs
- opt_out_phrases

Rep Assignment & Handoff

WF4 · CALLED BY WF3

- assignment_counter
- reps
- leads
- sequence_enrollments
- lead_events
- error_logs
- workflow_logs

No-Show Recovery

8 workflows · appointments

The largest service. Ingests bookings, sends reminders, detects missed appointments after a grace period, and runs a recovery sequence to get the patient rebooked.

Appointment Ingestion

NSR-01 · WEBHOOK

- appointments
- lead_events
- error_logs
- leads
- client_config

Reminders

NSR-02 · EVERY 10 MIN

- appointments
- lead_events
- error_logs
- sla_config
- client_config

No-Show Detection

NSR-03 · EVERY 5 MIN

- appointments
- lead_events
- error_logs

Recovery Sequence

NSR-04 · WEBHOOK

- sequence_enrollments
- sequence_touches
- appointments
- lead_events
- error_logs
- leads
- sequences
- client_config

Reply Detection & Handoff

NSR-05 · WEBHOOK

- opt_outs
- leads
- sequence_enrollments
- sequence_touches
- lead_events
- appointments
- reps
- opt_out_phrases

Sequence Ticker

NSR-06 · EVERY 15 MIN

- sequence_touches
- sequence_enrollments
- lead_events
- error_logs
- leads
- appointments
- sequences
- sla_config
- client_config

After-Hours Dispatcher

NSR-07 · EVERY 15 MIN

- sequence_touches
- sequence_enrollments
- lead_events
- error_logs
- leads
- sla_config

Booking Dashboard

NSR-08 · READ ONLY

- appointments
- leads

Quote & Proposal Follow-Up

5 workflows · sent proposals

Every proposal sent must end in won, lost with a reason, or sequence exhausted.
Triggered by a rep moving a lead into "Proposal Sent" in the sheet.

Quote Trigger Handler

QPF-1 · WEBHOOK FROM SHEETS

- sequence_enrollments
- sla_tracking
- lead_events
- error_logs
- Leads

Sequence Executor

QPF-2 · EVERY 15 MIN

- sequence_touches
- sequence_enrollments
- lead_events
- error_logs
- Leads
- reps
- sequences
- opt_outs

Reply Handler

QPF-3 · WEBHOOK

- opt_outs
- Leads
- sequence_enrollments
- lead_events
- reps
- opt_out_phrases

SLA Monitor

QPF-4 · EVERY 3 MIN

- sla_tracking
- heartbeat
- lead_events
- error_logs
- reps

Logging & Error Handler

QPF-5 · EVERY 5 MIN

- error_logs
- workflow_logs
- sequence_enrollments
- lead_events
- Leads

Review Agent

3 workflows · post-treatment

After a treatment is completed, waits a configured delay, asks the patient how it went, sends happy patients to the Google review page and unhappy ones straight to the clinic manager.

Completion Trigger, Poller & Response Handler

RV-01A / 01B / 01C

- appointments
- lead_events
- client_config
- reps

Voice Agent

2 workflows · inbound calls

The AI receptionist. Looks a caller up mid-conversation, creates them if new, and pulls a human in when the call needs one.

Tool Call Handler

AIR-01 · WEBHOOK

- leads
- lead_events
- error_logs

Human Handoff Notifier

AIR-02 · WEBHOOK

- leads
- lead_events
- error_logs
- reps

☒ Writes to it

☐ Reads from it only

Two things worth flagging

Attribution isn't wired yet. The ad-attribution table exists and the pipeline reads it, but no workflow currently writes to it — the capture contract has been handed over and still needs building into the intake and click-to-WhatsApp paths. Until then, per-ad performance can't be reported.

The knowledge base and FAQ tables are unused so far. Both are built and ready, but the intake bot doesn't yet read from either. It answers from its menu and question set only.

Rules that hold across every table

These aren't per-service decisions. They apply everywhere, and a few of them are enforced by the database itself rather than by discipline.

Phone numbers are never stored

Phone and email are scrambled into a one-way fingerprint before they ever reach the database. We can tell whether two enquiries are the same person; we cannot read the number back out. The real number lives only with the messaging provider.

Nothing is ever deleted

Records change status rather than disappearing — a stopped sequence is marked stopped, not removed. The timeline is strictly append-only, which is what makes any past week reproducible.

Human work is marked as human

Every logged event records whether a person did it or the system did. It's a single flag, and it's the reason our reports can say what the clinic actually did rather than what our automations did on their behalf.

One database per clinic

Every clinic gets their own Supabase project running the identical schema. Isolation is a property of the infrastructure, not of a filter someone might forget to apply — one clinic's data cannot appear in another's report even by accident.

Money stays internal

Deal values and revenue leakage are stored and used to diagnose problems, but stripped from anything the client sees. That separation is enforced by a test, not by a setting someone could flip.

Duplicates are blocked at the database

One lead per phone number, one opt-out per person, one touch per position in a sequence, one scorecard per clinic per week. These are database-level guarantees, so a workflow retrying itself can't produce a double.

Nothing public can read anything

Row-level security is on across every table with no public access rules attached, so anything short of a backend key is denied by default. Only our own pipeline and workflows can read.

Every table has a flexible pocket

Leads, events, messages and attribution each carry an open field for whatever a particular source brings along. New data from a new integration doesn't require a schema change — which matters because the database can't be altered from a workflow.

What we deliberately don't store

- Raw phone numbers or email addresses — only their fingerprints
- Any payment or card details, at any point in the flow
- Clinical or medical records — we hold the treatment enquired about, never a diagnosis or history
- Invoices or received payments — deal value is what an enquiry was *worth*, not revenue booked
- Full chat transcripts — we store what happened and how it was classified, not every word
- Anything belonging to another clinic — a separate database per client makes it structurally impossible